DIRECTORATE OF ECONOMICS AND STATISTICS

Data Management Portal

Technical Documentation & Interview Guide

Project	DES Data Management Portal
Stack	Flask (Python) · MySQL · Vanilla JS · HTML/CSS
Modules	Admin · Approver · User · Grievance · Statistics
Purpose	Multi-role Excel register upload, approval & reporting system

Table of Contents

1. System Overview

A multi-role web portal for the Directorate of Economics & Statistics to manage Excel register uploads, approvals and reporting.

Purpose

The DES Data Management Portal digitises the collection of statistical registers from departments across Tamil Nadu. Each department uploads Excel files containing census and survey data. These uploads pass through an approval workflow before being committed to the central database. The portal provides real-time statistics, grievance management, and granular permission control.

Tech Stack

Layer	Technology	Role
Backend	Flask (Python 3.11)	REST API, session auth, business logic
Database	MySQL 8	All data, permissions, uploads, attachments
Frontend	Vanilla JS + HTML/CSS	SPA-style dashboards, Chart.js charts
Excel	pandas + openpyxl	Template download, data upload, export
PDF	ReportLab	Documentation generation
Auth	Flask session + bcrypt	Role-based access control

Role Hierarchy

Admin	Full control — user management, permissions, statistics, grievance resolution
Approver	Reviews uploads — accepts/rejects rows, adds remarks, approves/rejects uploads
User	Uploads Excel registers, views status, edits rejected rows, submits grievances

2. Admin Module

Central control panel — user CRUD, permission management, upload monitoring, statistics & grievance resolution.

Key Capabilities

- Add / edit / delete users with department and role assignment
- Fine-grained permission control per user per department (View / Edit / Download)
- View all uploads across all departments with status badges
- Statistics tab: department completion rates, register-wise breakdown with dropdown filter
- Error log viewer with traceback display
- Grievance ticket dashboard — update status (Submitted → Pending → Closed) with resolution notes

Permission System

Permissions are stored per user per department. Three levels exist — they are not additive flags but represent intent:

Toggle	What it grants	Auto-rules
■ View	Read-only access to uploaded data	Removing View also removes Edit
✏ Edit	View + edit rows + delete rows + download	Granting Edit auto-enables View & Download
■ Download	Export data as Excel only	Removing Download also removes Edit

Backend — save permissions (admin_routes.py)

```
@admin_bp.route("/permissions/update", methods=["POST"])
def update_permissions(cursor, conn):
data = request.get_json()
user_id = data["user_id"]
perms = data["permissions"] # { "GLOBAL": {view,edit,download}, ... }
cursor.execute("DELETE FROM user_permissions WHERE user_id=%s", (user_id,))
for dept, p in perms.items():
cursor.execute("""INSERT INTO user_permissions
(user_id, department, can_view, can_edit, can_download)
VALUES (%s,%s,%s,%s,%s)""",
(user_id, dept, p["view"], p["edit"], p["download"]))
conn.commit()
return jsonify({"message": "Permissions saved"})
```

3. Approver Module

Reviews uploads from departments — approves or rejects at both upload and row level with a structured workflow.

Approval Workflow

Step	Action	Status Change
1	Department uploads Excel	Upload → Pending
2	Approver opens upload	Reviews each row
3	Row-level Accept / Reject	Row: is_approved = 1 or 0
4	Approver adds row remark	Row gets rejection explanation
5	All rows reviewed	Finalize Approval button shown
6	Approver finalizes	Upload → Approved or Rejected
7	User sees result	Green/Red badge on dashboard

Safe Bulk Approval

A key design decision: clicking 'Select All Accept' does NOT immediately approve the upload. It only marks all rows as accepted in the database and shows a yellow confirmation banner. The approver must explicitly click 'Finalize Approval'. If any row is later rejected, the upload status automatically reverts to Pending.

Frontend — finalizeUploadApproval() (view_excel_data.html)

```
async function finalizeUploadApproval() {
// Safety check before calling approve_upload
const anyRejected = allRowsData.some(r =>
approvalStatus(r["is_approved"]) === 0);
if (anyRejected) {
flash("Cannot approve — some rows are Rejected");
return;
}
await fetch("/api/approve_upload", {
method: "POST",
body: JSON.stringify({ upload_id: parseInt(uploadId) })
});
}
```

Backend — revert to pending when row is rejected (approver_routes.py)

```
@approver_bp.route("/api/revert_upload_pending", methods=["POST"])
@with_db_connection
def revert_upload_pending(cursor, conn):
upload_id = request.get_json()["upload_id"]
cursor.execute(
```

```
"UPDATE excel_uploads SET status_ = NULL WHERE id = %s",
(upload_id,)
)
conn.commit()
return jsonify({"message": "Reverted to pending"})
```

Row Remarks

When a row is rejected, the approver can add a free-text remark explaining why. Remarks are stored in a separate table linked to the row PK and upload ID. Users see these remarks as inline badges and can open a modal to read the full explanation.

```
DB — row_remarks table
CREATE TABLE row_remarks (
id INT AUTO_INCREMENT PRIMARY KEY,
upload_id INT NOT NULL,
table_name VARCHAR(100) NOT NULL,
row_id INT NOT NULL,
remark TEXT,
created_by VARCHAR(100),
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

4. User Module

Department staff upload Excel registers, monitor approval status, and re-edit rejected data.

Key Features

- Select register from dropdown → download the Excel template with correct columns
- Upload filled template — file is validated and stored with upload tracking
- Upload History tab shows all submissions with filter tabs (All / Pending / Approved / Rejected)
- Approved uploads show a Lock icon — no further edits allowed
- Rejected uploads show an Edit Rejected button — user edits only the rejected rows
- View Row Remarks button shows the approver's row-level explanations
- Request Re-Approval button resubmits corrected rows to the approver queue
- Submit Grievance option in the user dropdown menu

Ticket Number Format

Upload ticket number: `AGR-2026-03-0001` where `AGR` = first 3 letters of department, `2026-03` = year-month, `0001` = 4-digit running sequence (resets each month per department).

5. Statistics Module

Real-time dashboard showing department-level and register-level completion rates.

Department Details Tab

- Shows all departments with status: Submitted / Pending / Not Submitted
- Color-coded rows: green = approved, yellow = pending, red = not submitted
- Inline progress bar showing completion percentage per department
- Live search filter — type to instantly narrow the department list

Register-wise Breakdown Tab

Select a department from the dropdown. The system queries txn_registry for all registers assigned to that department and matches each against excel_uploads to determine status.

```
Backend — dept_register_breakdown (statistics_routes.py)
@statistics_bp.route("/api/statistics/dept_register_breakdown")
@with_db_connection
def dept_register_breakdown(cursor, conn):
dept = request.args.get("department")
# Get all registers for this dept from txn_registry
cursor.execute("""SELECT report_name, target_table_name
FROM txn_registry WHERE department = %s""", (dept,))
registers = cursor.fetchall()
for reg in registers:
# Count uploads and get latest status
cursor.execute("""SELECT COUNT(*) AS cnt, status_,
updated_date, updated_by FROM excel_uploads
WHERE department=%s AND table_name=%s
ORDER BY updated_date DESC LIMIT 1""",
[dept, reg["target_table_name"]])
upload = cursor.fetchone()
# status: approved / rejected / pending / not_uploaded
```

Summary Cards

Card	Calculation
Total Registers	COUNT(*) from txn_registry WHERE department = selected
Approved	Rows where upload status_ = 1
Pending	Rows where upload exists but status_ = NULL
Not Uploaded	Registers with no matching row in excel_uploads
Completion %	(Approved / Total) * 100

6. Grievance Handling Module

Users and approvers submit support tickets. Admin resolves and closes them with activity logging.

Database Design

- `tbl_grievance_type` — Master — 6 grievance categories (Download Excel, Upload Excel, Approval, etc.)
- `tbl_grievance_status` — Master — Submitted / Pending / Closed with color codes
- `tbl_grievance_ticket` — Transaction — every ticket: ticket_no, details, status, admin_remark
- `tbl_grievance_attachment` — Stores image/PDF as LONGBLOB in MySQL, linked to ticket
- `tbl_grievance_log` — Audit trail — every status change logged with timestamp and actor

Ticket Number Generation

grievance_routes.py — generate_ticket_no()

```
def generate_ticket_no(cursor, department=""):
# AGR-2026-03-0001 format
dept_code = (department[:3].upper() if department else "GEN")
prefix = f"{dept_code}-{datetime.now().strftime('%Y-%m')}-"
cursor.execute(
"SELECT COUNT(*) AS cnt FROM tbl_grievance_ticket
WHERE ticket_no LIKE %s", (f"{prefix}%",))
cnt = cursor.fetchone()["cnt"]
return f"{prefix}{str(cnt + 1).zfill(4)}"
# Result: AGR-2026-03-0001, AGR-2026-03-0002 ...
```

Attachment Storage

Attachments (screenshots, PDFs) are stored directly as LONGBLOB in MySQL — no filesystem dependency. When viewing, the backend fetches the bytes and returns them as base64 JSON. The frontend renders images inline or PDFs in an iframe popup.

Backend — return attachment as base64 (grievance_routes.py)

```
@grievance_bp.route("/api/grievance/attachment/")
@with_db_connection
def download_attachment(cursor, conn, ticket_id):
cursor.execute(
"SELECT filename, mimetype, file_data",
"FROM tbl_grievance_attachment WHERE ticket_id=%s LIMIT 1",
(ticket_id,))
row = cursor.fetchone()
if request.args.get("mode") == "base64":
import base64
b64 = base64.b64encode(row["file_data"]).decode("utf-8")
return jsonify({"filename": row["filename"],
"mimetype": row["mimetype"],
```

```
"data": b64})
```

Status Lifecycle

Status	Meaning	Who sets it
Submitted	Ticket just raised by user	System on submit
Pending	Admin is investigating	Admin — Update modal
Closed	Issue resolved, ticket locked	Admin — with resolution note

7. Database Design

MySQL schema with master tables for lookups and transaction tables for data.

Master Tables

- `users` — All users — username, password hash, department, role
- `tbl_departments` — Department list — name, code
- `txn_registry` — Register catalogue — report_name, target_table_name, department
- `tbl_column_metadata` — Display labels per column per table — used for Excel headers
- `tbl_grievance_type` — Grievance categories
- `tbl_grievance_status` — Ticket status definitions

Transaction Tables

- `excel_uploads` — Each upload attempt — filename, table_name, department, status_
- `tbl_livestock_census` — Example register — one row per submission, upload_id FK
- `row_remarks` — Per-row rejection notes from approver
- `user_permissions` — Per-user per-department access flags
- `tbl_grievance_ticket` — Grievance tickets
- `tbl_grievance_attachment` — Binary attachments linked to tickets
- `tbl_grievance_log` — Activity audit trail for tickets
- `error_logs` — System error logs for debugging

Key schema — excel_uploads

```
CREATE TABLE excel_uploads (
id INT AUTO_INCREMENT PRIMARY KEY,
filename VARCHAR(255),
table_name VARCHAR(100), -- which register
department VARCHAR(150),
uploaded_by VARCHAR(100), -- username
status_ TINYINT, -- NULL=pending 1=approved 0=rejected
rejection_reason TEXT, -- upload-level remark from approver
updated_date DATETIME
);
```

8. Key Code Snippets

The most interview-relevant patterns in the codebase.

DB Connection Decorator

All routes use a @with_db_connection decorator that injects cursor and conn:

app/utils/database.py

```
def with_db_connection(f):
@wraps(f)
def decorated(*args, **kwargs):
conn = get_connection() # from config
cursor = conn.cursor(dictionary=True)
try:
return f(cursor, conn, *args, **kwargs)
finally:
cursor.close()
conn.close()
return decorated
```

Dynamic PK Resolution

Each register table has a different primary key column. We query information_schema to find it:

row_remark_routes.py

```
def _get_pk_column(cursor, table):
cursor.execute("""
SELECT COLUMN_NAME FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_NAME=%s AND CONSTRAINT_NAME='PRIMARY'
LIMIT 1""", (table,))
row = cursor.fetchone()
return row["COLUMN_NAME"] if row else "id"
```

Excel Upload Processing

data_routes.py — upload_transaction()

```
@data_bp.route("/upload_transaction", methods=["POST"])
@with_db_connection
def upload_transaction(cursor, conn):
file = request.files["file"]
table_name = request.form["table_name"]
df = pd.read_excel(file)
# Map display labels back to column names
reverse_map = {v:k for k,v in col_to_label.items()}
df.rename(columns=reverse_map, inplace=True)
```

```
# Insert rows + create excel_uploads record
upload_id = create_upload_record(cursor, conn, ...)
df["upload_id"] = upload_id
df.to_sql(table_name, conn_alchemy, if_exists="append")
```

9. Common Interview Questions & Answers

Q: How does the approval workflow work?

A: Departments upload Excel files. Approvers see them in a queue. They review each row using Accept/Reject toggles. After reviewing all rows, they click 'Finalize Approval' which calls /api/approve_upload. If any row is rejected, the upload stays pending. The user is notified and can edit rejected rows and re-request approval.

Q: Why is a confirmation step needed before bulk approval?

A: Without it, clicking 'Select All Accept' would immediately lock the upload as Approved. If the approver then rejected a row, the row would be rejected in the data table but the upload would still show as Approved in the dashboard — an inconsistent state. The two-step design (mark + finalize) ensures consistency.

Q: How do you handle dynamic primary keys across different register tables?

A: Each register table has a different PK column name (e.g. livestock_census_refno). We query information_schema.KEY_COLUMN_USAGE at runtime to discover the PK column, then use it dynamically in all UPDATE and DELETE queries.

Q: How are attachments stored?

A: As LONGBLOB in MySQL in the tbl_grievance_attachment table. No filesystem is involved, so the application works identically on any server without directory configuration. For display, bytes are base64-encoded and sent as JSON — images render in an img tag, PDFs in an iframe.

Q: How is the permissions system designed?

A: Permissions are stored in user_permissions (user_id, department, can_view, can_edit, can_download). Edit is a superset — granting it auto-enables view and download. GLOBAL overrides apply to all departments not explicitly listed. The backend checks permissions on every route; the frontend shows/hides buttons accordingly.

Q: How does the statistics module know register completion?

A: txn_registry lists all registers per department. For each register, we query excel_uploads matching department + table_name. If no row exists, status = not_uploaded. If a row exists, we read status_ (NULL=pending, 1=approved, 0=rejected). The completion % = approved count / total register count for that department.

Q: How are grievance ticket numbers generated?

A: Format: DEP-YYYY-MM-NNNN. DEP = first 3 letters of the submitter's department. The sequence is per-department per-month, reset each month by counting existing tickets with that prefix. Example: AGR-2026-03-0001 for the first Animal Husbandry ticket in March 2026.

Q: How do you prevent the 'Missing table parameter' error on back navigation?

A: The view_excel_data page uses Jinja2 template variables like {{ table }}. browser history.back() replays the cached HTML without re-rendering the template, so {{ table }} appears literally. The fix: goBack() always navigates by role (/approver_dashboard, /user_dashboard etc.) instead of using history.back().

Directorate of Economics and Statistics

Data Management Portal — Technical Documentation

Confidential — For Interview Use Only